

Architecture & Migration Strategy

1. Canonical Entity Design Rationale

Background

The Cards, Transfers, and Bill Payments squads currently produce payment events using different schemas, field naming conventions, and status values. This creates challenges for analytics, reporting, fraud monitoring, and future product integrations.

To address this, I designed a canonical payment event model that standardizes common payment attributes across all product domains.

Reason for this Schema Structure

The schema focuses on the attributes that are consistently required across all payment products:

- event_id
- payment_type
- customer_id
- amount
- currency
- event_timestamp
- payment_method
- status
- source_system
- contract_version

These fields provide enough information to support operational reporting, customer analytics, payment monitoring, and downstream integrations without introducing product-specific complexity into the shared model.

Separating payment_type and payment_method was an intentional design decision. For example:

- payment_type = transfer
- payment_method = bank_transfer

This distinction allows future products to reuse the same payment type while supporting multiple execution methods.

Extensibility for Future Payment Types

The schema is designed to support future payment products without requiring significant redesign.

Potential future additions include:

- QR payments
- Merchant payments
- Wallet transfers
- International remittances
- Islamic financing repayments

New products can be introduced by adding new `payment_type` values while continuing to use the same core schema.

If product-specific attributes become necessary, they can be added through versioned contract extensions instead of modifying the core entity.

Trade-offs

To keep the initial rollout simple, I intentionally excluded:

- Merchant details
- Device information
- Risk scores
- Settlement information
- Fee breakdowns

Although these attributes may be useful in the future, including them in the first version would increase migration complexity and slow adoption across squads.

priority was achieving cross-team standardization quickly while leaving room for future expansion through schema versioning that can be used.

Architecture & Migration Strategy

1. Canonical Entity Design Rationale

Background

The Cards, Transfers, and Bill Payments squads currently produce payment events using different schemas, field naming conventions, and status values. This creates challenges for analytics, reporting, fraud monitoring, and future product integrations.

To address this, I designed a canonical payment event model that standardizes common payment attributes across all product domains.

Why I Chose This Schema Structure

The schema focuses on the attributes that are consistently required across all payment products:

- event_id
- payment_type
- customer_id
- amount
- currency
- event_timestamp
- payment_method
- status
- source_system
- contract_version

These fields provide enough information to support operational reporting, customer analytics, payment monitoring, and downstream integrations without introducing product-specific complexity into the shared model.

Separating payment_type and payment_method was an intentional design decision. For example:

- payment_type = transfer
- payment_method = bank_transfer

This distinction allows future products to reuse the same payment type while supporting multiple execution methods.

Extensibility for Future Payment Types

The schema is designed to support future payment products without requiring significant redesign.

Potential future additions include:

- QR payments
- Merchant payments
- Wallet transfers
- International remittances
- Islamic financing repayments

New products can be introduced by adding new `payment_type` values while continuing to use the same core schema.

If product-specific attributes become necessary, they can be added through versioned contract extensions instead of modifying the core entity.

Trade-offs

To keep the initial rollout simple, I intentionally excluded:

- Merchant details
- Device information
- Risk scores
- Settlement information
- Fee breakdowns

Although these attributes may be useful in the future, including them in the first version would increase migration complexity and slow adoption across squads.

My priority was achieving cross-team standardization quickly while leaving room for future expansion through schema versioning.

2. Phased Migration Plan

30-Day Phase: Foundation and Alignment

Objectives:

- Finalize canonical schema
- Publish Data Contract v1
- Review source mappings with all squads
- Build validation and transformation layer

Milestones:

- Schema approved by Cards, Transfers, and Bill Payments leads
- Mapping documentation completed
- Validation rules agreed upon
- Pilot environment established

At this stage, no production pipelines are modified.

The goal is alignment and stakeholder buy-in.

60-Day Phase: Pilot Rollout

Objectives:

- Onboard one squad to the canonical model
- Validate production compatibility
- Gather feedback before wider rollout

Adoption Sequence:

1. Bill Payments
2. Transfers
3. Cards

Rationale:

Bill Payments generally have the simplest payment flow and lowest dependency footprint, making them the lowest-risk candidate for initial migration.

Milestones:

- Bill Payments producing canonical events
- Validation dashboards operational
- Data quality metrics established
- Initial downstream consumers onboarded

Backward Compatibility Strategy:

During migration, squads continue publishing their existing schema while simultaneously publishing canonical events.

This dual-write strategy minimizes risk and prevents disruption to existing consumers.

90-Day Phase: Full Adoption

Objectives:

- Migrate remaining squads
- Establish shared governance model
- Begin deprecating legacy schemas

Milestones:

- Transfers migrated

- Cards migrated
- Shared contract ownership established
- Legacy schemas marked for retirement

Dependency Management

A dependency register will be maintained to identify:

- Analytics consumers
- Finance reporting users
- Fraud monitoring systems
- Regulatory reporting dependencies

No legacy schema will be removed until all known consumers have migrated successfully.

3. Data Contract & Governance

Versioning Strategy

The platform will use semantic versioning.

Non-breaking changes:

- Adding optional fields
- Adding new payment_type values
- Adding metadata attributes

Example:

v1.0 → v1.1

Breaking changes:

- Renaming fields
- Removing fields
- Changing data types

Example:

v1.x → v2.0

Schema Validation Enforcement

Validation will occur at three stages:

1. Source ingestion

2. Canonical transformation layer
3. Before writing to storage

Records failing validation will be redirected to an error dataset for investigation.

Ownership Model

Data Engineering owns:

- Canonical schema
- Validation framework
- Contract versioning

Product squads own:

- Source system mappings
- Data quality within their domain

Any schema changes require approval from:

- Data Engineering Lead
- Product Squad Representative
- Analytics Representative

4. Adoption Metrics & Stakeholder Plan

Success Metrics

The following KPIs will be tracked:

1. Percentage of payment volume using canonical schema
2. Number of squads onboarded
3. Reduction in downstream transformation logic
4. Data quality validation pass rate
5. Time required to onboard a new payment product

Target after 90 days:

- 100% payment coverage
- 95%+ validation success rate
- 50% reduction in duplicated transformation logic

Communication Plan

Weekly:

- Engineering sync meetings

Bi-weekly:

- Squad migration reviews

Monthly:

- Stakeholder progress updates

Documentation will be maintained centrally to ensure visibility across teams.

Managing Resistance

Some squads may prefer existing pipelines because they already support their operational needs.

To address this:

- Demonstrate reduced maintenance effort
- Provide migration support
- Use pilot success metrics as evidence
- Maintain backward compatibility during transition

The objective is collaboration rather than forcing adoption.

5. Production Considerations

Scaling to 100K Transactions per Day

For production deployment, I would introduce:

- Workflow orchestration using Airflow
- Partitioned Parquet storage
- Automated data quality monitoring
- Incremental processing
- Cloud object storage

The canonical schema itself would remain unchanged.

Monitoring and Alerting

Monitoring would include:

- Pipeline execution status

- Validation failure rates
- Data freshness
- Record volume anomalies
- Schema change detection

Alerts would be routed to engineering teams through operational monitoring tools.

What Was Intentionally Excluded

To keep Part 1 focused and deliverable within the assignment scope, I intentionally excluded:

- Real-time streaming ingestion
- Distributed processing frameworks
- Complex governance workflows
- Regulatory reporting integrations
- Advanced lineage tooling

These capabilities are important in production environments but would add unnecessary complexity to an initial proof-of-concept implementation.

The goal of Part 1 was to demonstrate schema standardization, contract-driven development, and a practical migration path across multiple payment products.